

Clasificación de género de películas

Daniel Montero, Simón Jaramillo, Juan Camilo Mejía

1. Preprocesamiento de los datos

Se separan los datos en la muestra de entrenamiento y prueba, y se convierten las clases en una matriz binaria usando `.fit_transform`, además se genera un embedding.

```
# Separación de muestras en train y test
X = dataTraining.drop(columns=['genres', 'year'])
y = dataTraining['genres']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
```

```
mlb = MultiLabelBinarizer()
y_train = mlb.fit_transform(y_train)
y_test = mlb.transform(y_test)
```

Se procede a descargar el embedding de BERT.

```
# Se usa el embedding pre cargado de BERT haciendo fine tuning

sbert_model = SentenceTransformer('all-mpnet-base-v2')
X_train_ = sbert_model.encode(X_train['plot'].tolist(), show_progress_bar=True)
X_test_ = sbert_model.encode(X_test['plot'].tolist(), show_progress_bar=True)
```

```
# Definición de dimensiones de salida, variables de interés
output_var = y_train.shape[1]
print(output_var, ' output variables')

# Definición de dimensiones de entrada, variables predictoras
dims = X_train_.shape[1]
print(dims, 'input variables')
```

24 output variables

1024 input variables

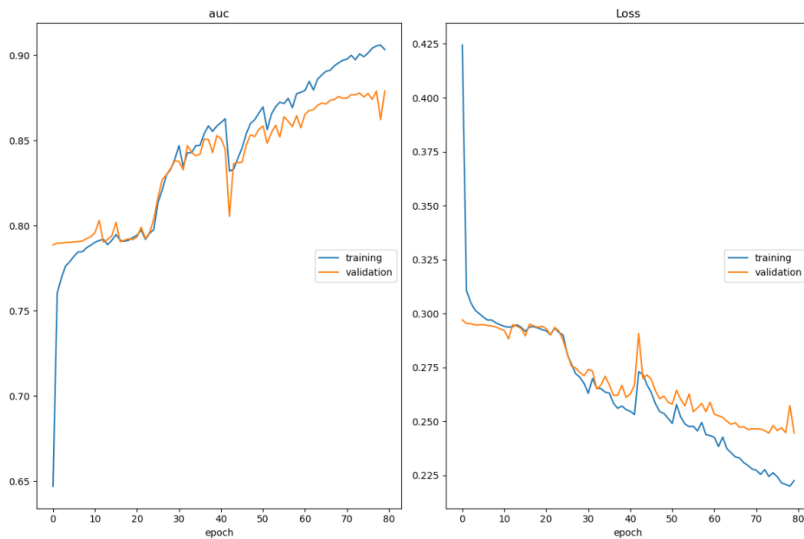
Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	1,000,000
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

Total params: 1,000,000 (3.81 MB)

Trainable params: 0 (0.00 B)

Non-trainable params: 1,000,000 (3.81 MB)



```
y_pred_test = model.predict(X_test_plot_pad)
roc_auc_score(y_test, y_pred_test, average='macro')
```

82/82 ————— 2s 29ms/step

0.7841531261594362

El uso del embedding ayudó bastante al modelo a no presentar un alto overfitting. Además, que se probaron varios emdeddings pero se decidió por el rendimiento y precisión el uso del embedding de BERT.

2. Calibración del modelo

Para la optimización de hiperparámetros se utilizó Keras, ya que está diseñado específicamente para modelos de redes neuronales además que es más eficiente que otros métodos como como gridsearch o randomsearch, además este puede tener varios inputs algo que no funciona bien con el grid, por estas razones se descartó el uso de estos métodos y se usó el de Keras.

```
def build_model(hp):
    K.clear_session()
    model = Sequential()
    # Capa de entrada
    model.add(Dense(units=hp.Int('units_1', min_value=64, max_value=256, step=64),
                      activation='relu',
                      input_shape=(X_train_.shape[1],)))
    model.add(Dropout(rate=hp.Float('dropout_1', 0.2, 0.5, step=0.1)))

    # Opcional segunda capa
    if hp.Boolean('second_layer'):
        model.add(Dense(units=hp.Int('units_2', 64, 256, 64), activation='relu'))
        model.add(Dropout(rate=hp.Float('dropout_2', 0.2, 0.5, step=0.1)))

    # Capa salida
    model.add(Dense(output_var, activation='sigmoid'))

    optimizer = Adam(learning_rate=hp.Float('lr', 1e-4, 1e-2, sampling='log'))
    model.compile(optimizer=optimizer,
                  loss='binary_crossentropy',
                  metrics=[tf.keras.metrics.AUC(name='auc')])

    return model
```

```
tuner = kt.RandomSearch(
    build_model,
    objective=kt.Objective("val_auc", direction="max"),
    max_trials=20,
    executions_per_trial=2,
    directory='my_dir',
    project_name='fasttext_model_opt'
)

early_stop = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

tuner.search(X_train_, y_train,
             epochs=100,
             validation_data=(X_test_, y_test),
             batch_size=32,
             callbacks=[early_stop],
             verbose=1)

# Mejor modelo
best_model = tuner.get_best_models(num_models=1)[0]

print(best_model.summary())
```

```
# Hiperparametros del mejor modelo
best_hyperparameters = tuner.get_best_hyperparameters(1)[0]
for hp in best_hyperparameters.values:
    print(f"{hp}: {best_hyperparameters.get(hp)}")
```

```
units_1: 192
dropout_1: 0.4
second_layer: False
lr: 0.00036448699004144517
units_2: 128
dropout_2: 0.30000000000000004
```

Parámetro	Valor óptimo	Cómo afecta al modelo
units_1	192	Número de neuronas en la primera capa oculta, este valor permite que la red aprenda patrones complejos desde el inicio, sin presentar sobrecargarla con demasiadas unidades.
dropout_1	0.4	Se descarta el 40 % de las neuronas de la primera capa durante el entrenamiento, esto ayuda a evitar el overfitting.
second_layer	False	Indica que no es necesaria una segunda capa oculta lo que simplifica el modelo, reduciendo el riesgo de overfitting.
lr	0.0003644	Tasa de aprendizaje muy pequeña, lo que permite ajustes muy finos en cada paso, también requiere más épocas para converger, pero ayuda a estabilizar el proceso de entrenamiento.
units_2	128	Número de neuronas que tendría la segunda capa (aunque second_layer=False, este valor podría ser parte de otra configuración previa o futura) y, si se usara, permitiría refinar aún más los patrones aprendidos.
dropout_2	0.30	Porcentaje de neuronas descartadas en la segunda capa (si existiera), un 30% es una regularización moderada que ayuda a evitar el overfitting sin afectar demasiado la capacidad de aprendizaje.

(aunque igual da la información de lo más óptimo en caso de que se quiera usar una segunda capa)

```
# Dado que la red mono capa dió mejor desempeño, se usará una red monocapa para modelar
K.clear_session()

model = Sequential()
model.add(Dense(192, activation='relu', input_shape=(dims,)))
model.add(Dropout(0.4))
model.add(Dense(24, activation='sigmoid'))

print(model.summary())
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 192)	196800
dropout (Dropout)	(None, 192)	0
dense_1 (Dense)	(None, 24)	4632

=====
 Total params: 201432 (786.84 KB)
 Trainable params: 201432 (786.84 KB)
 Non-trainable params: 0 (0.00 Byte)

None

```
# Lr = 0.0003127768409735319
optimizer = tf.keras.optimizers.Adam(learning_rate= 0.00036448699004144517)
model.compile(optimizer=optimizer, loss='binary_crossentropy', metrics=[tf.keras.metrics.AUC(name='auc')])
early_stop=EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

modelo=model.fit(X_train_, y_train,
                 validation_data=(X_test_, y_test),
                 verbose=1,
                 epochs=100,
                 batch_size=32,
                 callbacks=[early_stop])
```

on metric `val\_loss` which is not available. Available metrics are: loss, auc
 247/247 [=====] - 0s 1ms/step - loss: 0.1451 - auc: 0.9636

```
# Evaluación del modelo en test
y_pred_test = model.predict(X_test_)
roc_auc_score(y_test, y_pred_test, average='macro')
```

82/82 [=====] - 0s 723us/step
 0.9260149351912269

Igualmente se probaron varios tipos de transformación de los datos, tales como concatenar el título con la descripción, dejarlos separados, generar un campo calculado para la fecha mostrando la antigüedad, pero por no mostrar mejoras en el desempeño se llegó a la conclusión que el desempeño más óptimo era solamente con el uso del embedding.

3. Entrenamiento del modelo

Se entrenaron varios modelos para la evaluación del género de las películas, estos fueron una red neuronal monocapa, una red neuronal multicapa y una red neuronal LSMT, a lo que se decidió usar la red neuronal monocapa ya que cuando se complejizaba el modelo se disminuía el desempeño, además con el embedding

Red Neuronal Mono Capa

```
vect_plot = CountVectorizer(max_features=5000)
vect_title = CountVectorizer(max_features=5000)
plot_train = vect_plot.fit_transform(X_train['plot']).toarray()
plot_test = vect_plot.transform(X_test['plot']).toarray()
title_train = vect_title.fit_transform(X_train['title']).toarray()
title_test = vect_title.transform(X_test['title']).toarray()
```

```
X_train_mc = np.hstack((plot_train, title_train))
X_test_mc = np.hstack((plot_test, title_test))
```

```
# Definición de dimensiones de salida, variables de interés
output_var = y_train.shape[1]
print(output_var, ' output variables')

# Definición de dimensiones de entrada, variables predictoras
dims = X_train_mc.shape[1]
print(dims, 'input variables')
```

24 output variables
10000 input variables

```
K.clear_session()

# Definición red neuronal con la función Sequential()
model = Sequential()
model.add(Dense(256, input_shape=(dims,), activation='relu'))
model.add(Dense(256, activation='relu'))
model.add(Dense(output_var))
model.add(Activation('sigmoid'))

# Impresión de la arquitectura de la red neuronal
print(model.summary())
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 256)	2,560,256
dense_1 (Dense)	(None, 256)	65,792
dense_2 (Dense)	(None, 24)	6,168
activation (Activation)	(None, 24)	0

Total params: 2,632,216 (10.04 MB)
Trainable params: 2,632,216 (10.04 MB)
Non-trainable params: 0 (0.00 B)
None

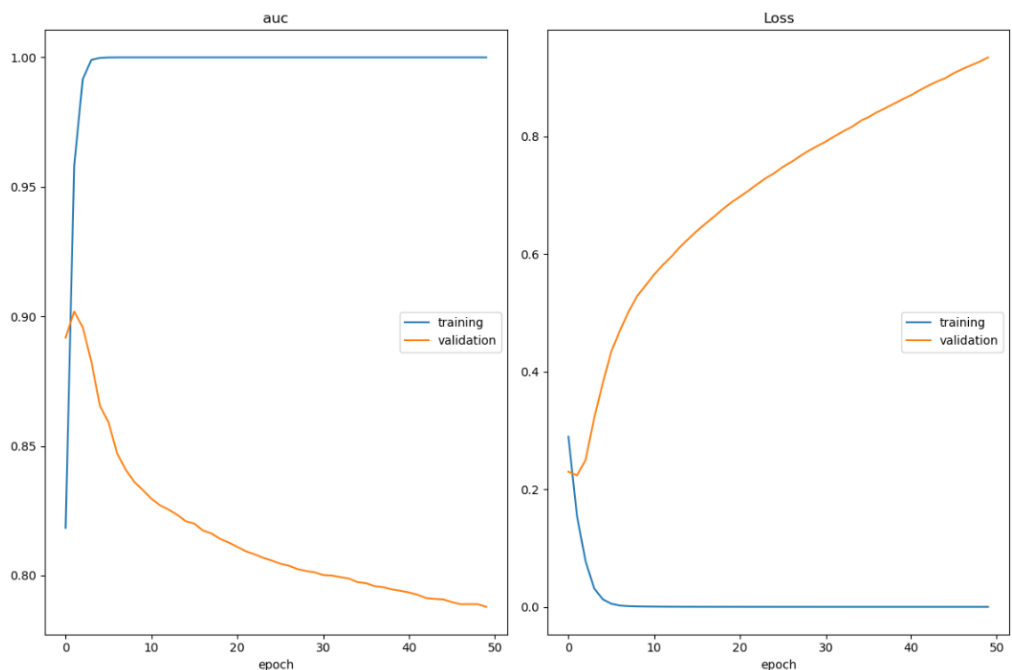
```
model.compile(optimizer='adam', loss='binary_crossentropy',metrics=[tf.keras.metrics.AUC(name='auc')])
```

```
model.fit(X_train_mc, y_train,
        verbose=1,
        epochs=100,
        validation_data=(X_test_mc, y_test),
        callbacks=[PlotLossesKeras()])
```

```

auc
      training      (min: 0.818, max: 1.000, cur: 1.000)
      validation    (min: 0.788, max: 0.902, cur: 0.788)
Loss
      training      (min: 0.000, max: 0.289, cur: 0.000)
      validation    (min: 0.224, max: 0.934, cur: 0.934)
166/166 ————— 3s 16ms/step - auc: 1.0000 - loss: 1.6023e-06 - val_auc: 0.7879 - val_loss: 0.9342
<keras.src.callbacks.history.History at 0x1f1ee260610>

```



```
y_pred_test = model.predict(X_test_mc)
roc_auc_score(y_test, y_pred_test, average='macro')
```

```

82/82 ————— 0s 2ms/step
0.8371387166651892

```

Se muestra un desempeño aceptable aun cuando se presenta overfitting.

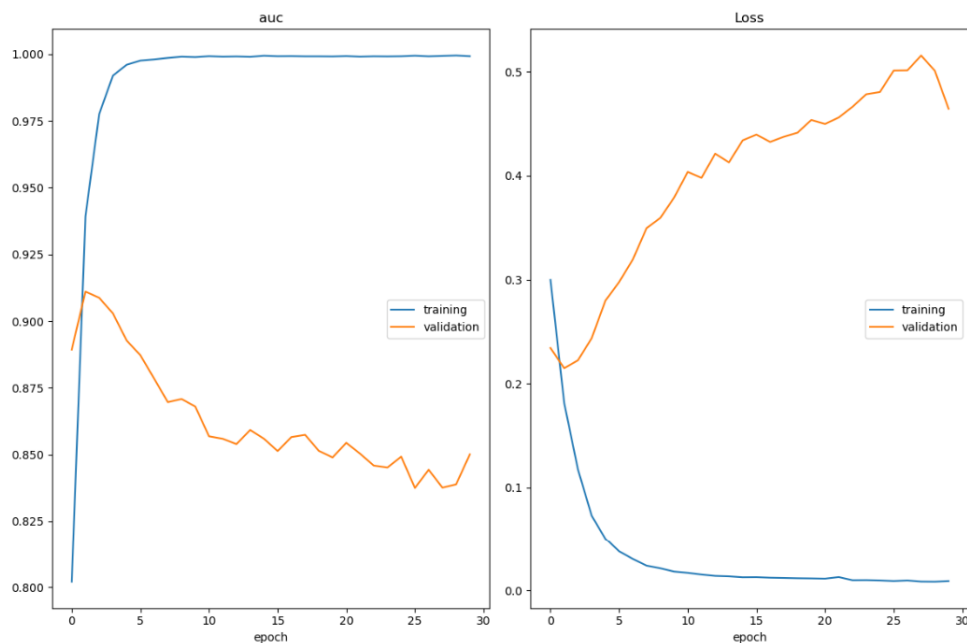
Red Neuronal Multicapa

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	5,120,512
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 24)	6,168

```

Total params: 5,258,008 (20.06 MB)
Trainable params: 5,258,008 (20.06 MB)
Non-trainable params: 0 (0.00 B)

```



Epoch 12/100

166/166 ————— **6s** 34ms/step - auc: 0.9991 - loss: 0.0163 - val_auc: 0.8600 - val_loss: 0.4068

```
y_pred_test = model.predict(X_test_mc)
roc_auc_score(y_test, y_pred_test, average='macro')
```

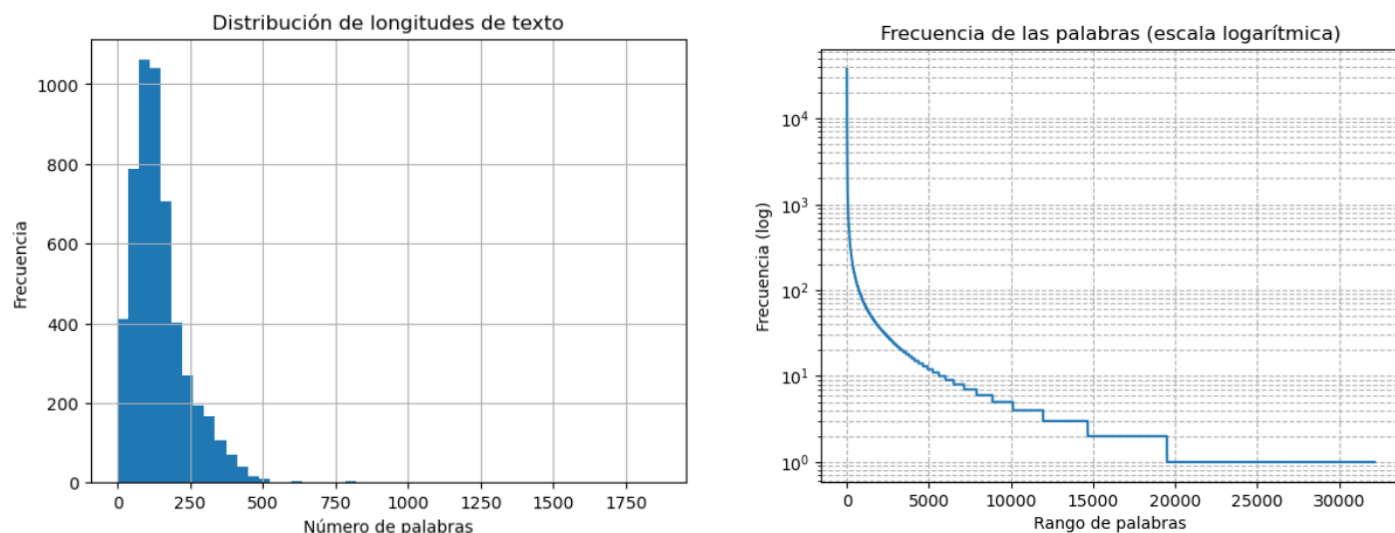
82/82 ————— **0s** 2ms/step

0.8458715535207514

Se puede evidenciar que, aunque el ROC AUC es aceptable, se presenta un alto overfitting desde las primeras épocas, también el aumento que se tuvo desde la mono capa es mínimo y lo que se logró fue una complejización del modelo.

Red Neuronal LSTM

Para conocer los parámetros óptimos para el modelo se realizan las siguientes gráficas las cuales muestran la distribución según la longitud del texto como el de la frecuencia de las palabras.



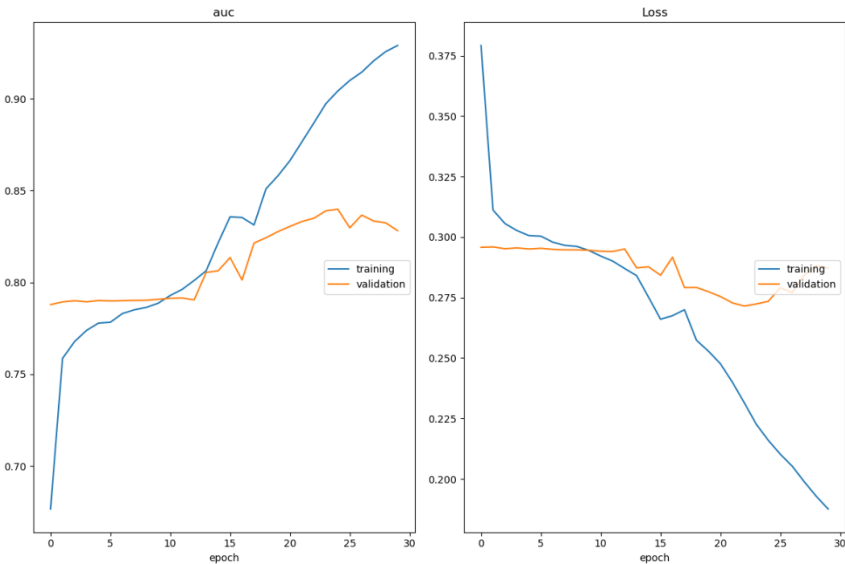
Con base en estas gráficas, se definieron los siguientes valores para los parámetros del modelo:

```
# Parámetros
vocab_size = 20000
maxlen = 400
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)



```
y_pred_test = model.predict(X_test_plot_pad)
roc_auc_score(y_test, y_pred_test, average='macro')
```

82/82 ————— 3s 30ms/step
0.7144769392309849

No se demuestra un alto desempeño del modelo, además de mostrar la aparición de overfitting en los datos.

4. Disponibilización del modelo

Para la visualización de la instancia visitar <http://3.136.97.13:5000/>

aws

Buscar

[Alt+S]

Estados Unidos (Ohio)

simon.jaramillo

```
$20*$20to$20care$20if$20the$20is$20guilty$20or$20innocent$20,$20$20but$20needs$20to$20know$20the$20truth$20to$20defend$20him$20adequately$20,$20$20unlike$20the$20rest$20of$20the$20world$20,$20$20many$20do$20not$20believe$20his$20story$20,$20$20he$20who$20hopes$20he$20can$20use$20aaron$20$20$20general$20appearance$20of$20being$20an$20innocent$20to$20his$20advantage$20,$20$20the$20powerful$20state$20attorney$20,$20$20john$20$20$20haughnessy$20,$20$20who$20marry$20has$20had$20many$20a$20moral$20run$20-$20in$20,$20$20wants$20a$20first$20degree$20murder$20conviction$20and$20the$20death$20penalty$20in$20this$20case$20,$20$20the$20appoints$20to$20the$20case$20janet$20venable$20,$20$20who$20s$20till$20has$20bad$20relationships$20towards$20marry$20,$20$20an$20ex$20$20lower$20,$20$20the$20s$20month$20relationship$20which$20ended$20badly$20,$20$20althe$20up$20the$20case$20looks$20to$20be$20a$20claim$20against$20janet$20,$20$20her$20arrest$20may$20be$20made$20as$20the$20case$20is$20outcom$20,$20$20$20in$20a$20building$20his$20case$20,$20$20marry$20comes$20across$20some$20major$20pieces$20of$20information$20,$20$20some$20pertaining$20to$20the$20archbishop$20himself$20,$20$20and$20one$20uncovered$20by$20dr$20,$20$20$20molly$20darrington$20about$20aaron$20,$20$20she$20a$20psy$20chiarist$20hired$20by$20marry$20to$20assess$20aaron$20*$20$20$20mental$20state$20,$20$20these$20pieces$20of$20information$20as$20a$20collecti$20ve$20piece$20a$20problem$20for$20marry$20to$20in$20how$20to$20mount$20a$20credible$20and$20legitimate$20defense$20for$20his$20client$20,$20$20$20it$20is$20more$20of$20a$20moral$20dilemma$20for$20marry$20if$20only$20because$20he$20believes$20the$20life$20of$20a$20young$20man$20,$20$20$20who$20he$20believes$20in$20,$20$20is$20at$20stake$20,$20$20ar HTTP/1.1" 200 -
clear
^C(myenv) ubuntu@ip-172-31-28-244:~/MIAD_ML_NLP_2025/Despliegue$ python api.py
* Serving Flask app 'api'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://172.0.0.1:5000
* Running on http://172.31.28.244:5000
Press CTRL+C to quit
```

i-0d0f79395a870a8d3 (ML_PLN_GRUPO_02)

PublicIPs: 3.136.97.13 PrivateIPs: 172.31.28.244


```
[ Base URI : / ]
/swagger.json
```

predict Genre Classifier

Genres movies Prediction API ^{1.0}

[Base URL: /]
/swagger.json

Genres movies Prediction API

[illegible]

- El modelo óptimo para este caso fue el de tener una red neuronal mono capa, ya que este además de ser más sencillo era el que mejor desempeño presentaba, incluso con los parámetros encontrados con Keras mostró que lo ideal para este caso era una sola capa.
- Aunque se probaron varios embeddings, los mejores resultados fueron obtenidos por medio de BERT, el cual fue usado solamente con el modelo mono capa, ya que realizar diferentes transformaciones, y usos de otros parámetros diferentes a los mencionados complejizaban el modelo sin mostrar mejora alguna en el desempeño, además de ayudar con la corrección del overfitting en los datos, por lo que en embedding en este caso fue la clave para un rendimiento óptimo.
- La experimentación con arquitecturas más complejas, como redes multicapa y LSTM, dejó en evidencia que una mayor complejidad no siempre se significa un mejor desempeño, de hecho, estas opciones incrementaron la aparición de overfitting sin mejoras significativas en la métrica AUC ROC.
- La optimización con Keras demostró ser eficaz al momento de ajustar los hiperparámetros de los modelos de redes neuronales, especialmente cuando se manejan entradas múltiples, este método permitió encontrar una configuración equilibrada entre capacidad de entrenamiento y validación.
- Aunque se probaron distintas transformaciones en los datos, como la combinación del título y la descripción o la inclusión creación de columnas como la fecha de antigüedad, estas no generaron mejoras en el desempeño del modelo, por el contrario, introdujeron ruido, por lo que se decidió descartarlas.